

JOBRIGHT MULTI-CATEGORY JOB SCRAPING & SALARY ANALYTICS SUITE

Technical & User Report

Scope: Data Scientist · Data Analyst · Machine Learning Engineer · Statistician

Prepared for the Jobright-suite project

July 2026

1. Executive Summary

This project is a self-hosted pipeline that scrapes live job postings from Jobright.ai for four target categories — Data Scientist, Data Analyst, Machine Learning Engineer, and Statistician — normalizes each posting's required major, job role, and seniority level into a consistent taxonomy, and presents the combined result through an interactive salary and demand dashboard.

The suite is fully modular: the scraper, the taxonomy mapper, the category list, the command-line tools, the desktop GUI, and the dashboard are each separate files that can be run independently. The list of job categories being tracked is not hardcoded — it lives in an editable JSON configuration file, so categories can be added or removed at any time without modifying code.

2. Purpose & Motivation

The tool exists to answer a practical question: across data-oriented career tracks, which required majors, job titles, and seniority levels are associated with the strongest pay and the highest posting volume right now? Rather than reading job postings one at a time, the pipeline aggregates hundreds of live listings per category, extracts salary ranges and degree requirements directly from each posting's detail page, and rolls that up into comparable statistics.

3. System Architecture

The pipeline runs in six stages, each owned by a specific file. Data flows one direction: configuration in, scraped Excel files out, then a combined Excel file, then the dashboard reads that combined file.

1. <code>input/categories_config.json</code> (editable list of job categories: key, label, URL, role tag)
2. <code>pipeline_categories.py</code> -> reads config, dispatches one scrape per category
3. <code>pipelinescrapper_core.py</code> -> Playwright opens Jobright category URL, scrolls, extracts job cards, opens detail panel for degree/major text, autosaves to <code>output/<category>/jobright_jobs.xlsx</code>
4. <code>canonical_postprocess.py</code> -> rewrites that Excel in place, adding canonical major / role / seniority columns
5. <code>merge_categories.py</code> -> combines all category Excel files into <code>output/jobright_jobs.xlsx</code> , tagged with <code>search_category</code> , de-duplicated by <code>job_url</code>
6. <code>dashboard_app.py</code> (Streamlit) -> reads the combined Excel, renders Main Dashboard / Branch Salary Summary / Overall Statistics tabs

4. File-by-File Reference

All eight files sit in the project's code/ folder alongside the project's original input/ (canonical majors list) and output/ (generated Excel files) directories.

File	Role	Key Responsibility
pipelinescraper_core.py	Scraping engine	Drives Playwright/Chromium against a Jobright category URL; extracts job cards, opens detail panels for degree/branch text, autosaves to Excel/JSONL, and resumes from state.json.
canonical_postprocess.py	Taxonomy mapping	Maps raw scraped text into canonical buckets: required major (branch), job role (role_name), and seniority level, using alias rules, wildcard patterns, and fuzzy matching.
pipeline_categories.py	Category orchestration	Loads/saves the category list from input/categories_config.json; runs one category end-to-end (scrape + postprocess); merges all categories into one Excel.
scrape_category.py	CLI entry point	Command-line interface to scrape, list, add, or remove categories without touching code.
merge_categories.py	CLI entry point	Thin wrapper that combines every category's Excel file into output/jobright_jobs.xlsx.
gui_runner.py	Desktop GUI	Tkinter window: category dropdown with Add/Remove dialogs, job-count selector, Start/Continue/Merge/Open Dashboard buttons, live log.
dashboard_app.py	Visualization	Streamlit app with three tabs: Main Dashboard, Branch Salary Summary, and Overall Statistics.
setup_and_run.py	Installer/launcher	Single entry point exposing install / scrape / merge / gui / dash subcommands.

5. Dynamic Category Configuration

Job categories are stored in input/categories_config.json, a plain JSON file mapping a short key to a label, a Jobright search URL, and a role tag used to fill the role_selected column. This file is created automatically on first run with four default categories.

Key	Label	Jobright URL
data_scientist	Data Scientist	jobright.ai/jobs/data-scientist-jobs-in-united-states
data_analyst	Data Analyst	jobright.ai/jobs/data-analyst-jobs-in-united-states

Key	Label	Jobright URL
machine_learning_engineer	Machine Learning Engineer	jobright.ai/jobs/machine-learning-engineer-jobs-in-united-states
statistician	Statistician	jobright.ai/jobs/statistician-jobs-in-united-states *

* At the time this report was written, this exact category URL could not be independently confirmed as a live standalone Jobright landing page — closely related searches (e.g. 'sr-statistician', 'phd-statistician') were confirmed live, which is a strong signal the URL slug pattern holds, but it should be verified by opening the URL directly before relying on it for a full scrape run.

5.1 Adding or removing categories

Categories can be changed three ways, all equivalent:

- Edit input/categories_config.json directly in any text editor.
- Command line: `python code\scrape_category.py add <key> "<Label>" "<url>" "<role>"`
- GUI: click the "+ Add" or "- Remove" buttons next to the category dropdown.

No code changes or restarts are required — the next scrape or dashboard load picks up the change automatically.

6. Data Extraction Details

6.1 What gets captured per job

For every job card, the scraper extracts: platform, role_selected (which category search produced it), role_name, role_type, work_model, company_name, company_url, job_url, salary (raw text) plus parsed salary_min / salary_max, degree_level, branch (required field of study, extracted from the full job description text via regex), location, posted date, and seniority.

6.2 Canonical mapping

Because job postings phrase things inconsistently (e.g. "Info Systems" vs. "Information Systems" vs. "MIS"), a separate postprocessing step maps each raw value into a fixed taxonomy:

- Branch (major): mapped against a user-maintained canonical list (input/canonical_majors_custom_full.xlsx) using alias rules, exact match, then fuzzy matching. Buckets relevant to this project's scope include Data Science, Statistics, Applied Mathematics, Computer Information Systems, Information Systems, Information Sciences, and Information Technology.
- Role: mapped into canonical role buckets — Data Analyst, Data Scientist, Data Engineer, Machine Learning Engineer, AI Engineer, Statistician, Business Analyst, Research Scientist, Quantitative Analyst, Software Engineer, Product Manager — using wildcard text patterns (e.g. "*statistician*") and short-token aliases (e.g. "mle", "ds", "quant").
- Seniority: mapped into Intern, Entry, Mid, Senior, Lead, Manager, Director, VP, and C-level using the same wildcard-pattern approach.

6.3 Resumability

Each category maintains its own state.json and jobs_raw.jsonl under output/<category>/. If a scrape is interrupted (crash, Ctrl+C, or you simply stop early), re-running the same category command picks up exactly where it left off rather than starting over or duplicating rows.

6.4 Login session reuse

The first category scraped in a session requires a manual login inside the opened browser window. That login session is saved to output/dom_snapshots/jobright_login_session.json and automatically reused for every subsequent category, so login is only needed once per machine (until the session expires).

7. Setup Instructions

1. Place all eight project files in the code/ folder alongside the existing input/ and output/ directories.
2. Open a terminal at the project root and install dependencies:

```
cd <project-root> python code\setup_and_run.py install
```

This installs pandas, openpyxl, playwright, streamlit, plotly, and beautifulsoup4, then downloads Playwright's Chromium browser binary.

8. Running the Pipeline

8.1 Command-line workflow

```
python code\setup_and_run.py scrape data_scientist 150 python code\setup_and_run.py scrape data_analyst 150 python code\setup_and_run.py scrape machine_learning_engineer 150 python code\setup_and_run.py scrape statistician 150 python code\setup_and_run.py merge python code\setup_and_run.py dash
```

8.2 GUI workflow

Run python code\setup_and_run.py gui to open the desktop application. Select a category and a job count, click Start Scraping, complete the login prompt once, then click Continue after login. Repeat for each category, click Merge All Categories, then click Open Dashboard.

9. Full Command Reference

Command	What it does
python code\setup_and_run.py install	Installs Python dependencies and Playwright's Chromium browser. Run once.
python code\setup_and_run.py scrape list	Lists every configured category key, label, and URL.

Command	What it does
<code>python code\setup_and_run.py scrape <key> <count></code>	Scrapes one category up to <count> jobs, autosaving as it goes; resumable if interrupted.
<code>python code\scrape_category.py add <key> "<Label>" "<url>" "<role>"</code>	Adds a new category to input/categories_config.json without editing code.
<code>python code\scrape_category.py remove <key></code>	Removes a category from the config (scraped data is not deleted).
<code>python code\setup_and_run.py merge</code>	Combines every category's Excel into output/jobright_jobs.xlsx, tagging rows with search_category and de-duplicating by job_url.
<code>python code\setup_and_run.py gui</code>	Launches the Tkinter desktop app (scrape / merge / dashboard in one window).
<code>python code\setup_and_run.py dash</code>	Launches the Streamlit dashboard in your browser.

10. Dashboard Overview

The Streamlit dashboard (output at <http://localhost:8501>) is organized into three tabs:

10.1 Main Dashboard

Displays the raw combined dataset with filters for search category, company, work model, role type, and seniority. Includes a branch-metrics bar chart (count or salary by canonical major) and an optional "Multi-X Chart" builder supporting bar, line, scatter, box, and histogram views with a lightweight natural-language chart suggester.

10.2 Branch Salary Summary

Generates an aggregated summary Excel (count, average/median salary, salary range, and standard deviation per canonical field) and renders it as configurable bar, pie, scatter, and comparison charts, with the option to split bars by degree level, seniority, search category, or detected seniority marker fields.

10.3 Overall Statistics

A dedicated statistics tab added specifically for this multi-category build. Shows headline KPI cards (total jobs, unique companies, average and median salary), a jobs-by-category bar and pie chart, a per-category salary comparison table and chart, an overall salary distribution histogram, work-model and seniority breakdown pie charts, and a top-20 hiring companies bar chart.

11. Known Limitations & Notes

- The Statistician category URL was not independently confirmed as a live standalone Jobright page at time of writing; verify it on first use and update the config if needed (see Section 5).

- Jobright requires an authenticated session to view job cards; scraping cannot proceed unattended without at least one manual login.
- Scrape speed depends on Jobright's page load and scroll behavior; the loop halts automatically after ~1500 scrolls with no new postings.
- The canonical major mapping quality depends on the completeness of input/canonical_majors_custom_full.xlsx — majors not present there and not close enough for fuzzy matching will not be bucketed.
- All scraping is scoped to United States postings, matching the category URLs currently configured.

12. Summary

This suite converts unattended browsing of a job board into a structured, repeatable research asset: four configurable job-category searches, normalized against a shared taxonomy, merged into one dataset, and explored through a purpose-built statistics dashboard — with the category list itself editable at any time without code changes.